Unidad 4 - Programación Orientada a Objetos (POO) - JA Alejo

- **Autor**: José Ángel Alejo Sillero
- **Centro**: Instituto Tecnológico Pablo de la Torre

- **Curso**: 2025/2026
- **Módulo**: Programación
- **Unidad**: 4 - Programación Orientada a Objetos (POO)

Tabla de Contenidos

Introducción a la Programación Orientada a Objetos (POO)

La Programación Orientada a Objetos (POO) es un paradigma de programación que utiliza "objetos" para representar datos y métodos. Este enfoque facilita la organización del código, la reutilización y el mantenimiento.

Cuando se trabaja con POO, se definen clases que actúan como plantillas para crear objetos. Cada objeto es una instancia de una clase y puede contener atributos (datos) y métodos (funciones).

Cuando indicamos que un método es "estático", significa que pertenece a la clase en sí misma y no a una instancia específica de esa clase. Los métodos estáticos se pueden llamar sin crear una instancia de la clase, es decir, sin crear un objeto de esa clase. Esto genera independencia entre el método y los objetos de la clase.

Hasta ahora hemos usado el método con el formato `public static void main(String[] args)`, que es un método estático que sirve como punto de entrada para la ejecución de un programa en Java.

El método se compone de varias partes:

- **Modificador**: Indica el nivel de acceso del método.
  - `public`: El método es accesible desde cualquier otra clase.
  - `static`: Indica que el método pertenece a la clase y no a una instancia específica.
- **Tipo de retorno**: Indica el tipo de dato que el método devolverá. En este caso, `void` significa que el método no devuelve ningún valor.
  - La función se comportará de acuerdo al tipo de retorno que se defina, habilitandonos a utilizar métodos de ese tipo de dato con nuestro método.
- **Nombre del método**: `main` es el nombre del método que se utiliza como punto de entrada para la ejecución del programa.
- **Parámetros**: `(String[] args)` es un arreglo de cadenas que permite pasar argumentos al programa desde la línea de comandos.

Ahora bien, el tipo de retorno puede variar según la función del método. Por ejemplo, si un método debe devolver un número entero, se utilizará `int` como tipo de retorno. Siempre que un método tenga un tipo de retorno distinto a `void`, debe incluir una instrucción `return` que devuelva un valor del tipo especificado.

En `return`, se indica el valor que el método devolverá al lugar donde fue llamado. Por ejemplo, si un método tiene un tipo de retorno `int`, la instrucción `return` debe devolver un valor entero. También se puede devolver objetos o valores de otros tipos de datos según lo especificado en la declaración del método. También es posible utilizar `return` sin un valor en métodos con tipo de retorno `void`, simplemente para salir del método antes de que termine su ejecución.

Por ejemplo, para devolver enteros:

- `return suma;` (donde `suma` es una variable de tipo `int`).
- `return 42;` (devolviendo un valor entero literal).
- `return n1 + n2;` (devolviendo el resultado de una expresión aritmética).

Dentro de nuestro método se pueden utilizar tres tipos de variables: Temporales, de Instancia y de Clase.

- **Variables Temporales**: Son aquellas que se declaran dentro de un método y solo existen durante la ejecución de ese método. Una vez que el método termina, estas variables se destruyen y su valor se pierde. Se utilizan para almacenar datos temporales necesarios para la lógica del método.
- **Variables de Instancia**: Son variables que pertenecen a una instancia específica de una clase. Cada objeto creado a partir de la clase tiene su propia copia de estas variables. Se utilizan para almacenar el estado o las propiedades del objeto.
- **Variables de Clase**: Son variables que pertenecen a la clase en sí misma y no a una instancia específica. Se declaran con la palabra clave `static` y son compartidas por todas las instancias de la clase. Se utilizan para almacenar datos que son comunes a todas las instancias.

Ejemplo de cada tipo de variable:

```
public class EjemploVariables {
    // Variable de Clase
    static int contadorInstancias = 0;

    // Variable de Instancia
    int id;

    public EjemploVariables() {
        // Incrementar el contador de instancias cada vez que se crea un nuevo objeto
        contadorInstancias++;
        // Asignar un ID único a la instancia
        this.id = contadorInstancias;
    }

    public void mostrarInfo() {
        // Variable Temporal
        String info = "ID de la instancia: " + id + ", Total de instancias: " + contadorInstancias;
        System.out.println(info);
    }

    public static void main(String[] args) {
        EjemploVariables obj1 = new EjemploVariables();
        obj1.mostrarInfo();

        EjemploVariables obj2 = new EjemploVariables();
        obj2.mostrarInfo();
    }
}
```

Los métodos o funciones no sólo se definen por su nombre, sino también por sus parámetros. La combinación del nombre del método y la lista de parámetros se conoce como "firma del método". Esto permite que una clase tenga múltiples métodos con el mismo nombre, siempre y cuando tengan diferentes listas de parámetros (diferente número o tipo de parámetros). Este concepto se llama "sobrecarga de métodos".

Ejemplo de sobrecarga de métodos:

```
import java.util.Scanner;

public class Calculadora {
    // Método para sumar dos enteros
    public int sumar(int a, int b) {
        return a + b;
    }

    // Sobrecarga del método sumar para sumar tres enteros
    public int sumar(int a, int b, int c) {
        return a + b + c;
    }

    // Sobrecarga del método sumar para sumar dos números de punto flotante
    public double sumar(double a, double b) {
        return a + b;
    }

    // Método para restar dos enteros
    public int restar(int a, int b) {
        return a - b;
    }

    // Método para calcular el valor absoluto de un entero
    public int absoluto(int a) {
        return Math.abs(a);
    }

    public static void main(String[] args) {
        Calculadora calc = new Calculadora();
        Scanner scanner = new Scanner(System.in);

        while (true) {
            System.out.print("\nMenú: \n  1. Suma\n  2. Resta\n  3. Valor absoluto\n  4. Salir\nElige una opción: ");
            int opcion = scanner.nextInt();
            int a, b;

            switch (opcion) {
                case 1 -> {
                    System.out.print("Ingresa primer número: ");
                    a = scanner.nextInt();
                    System.out.print("Ingresa segundo número: ");
                    b = scanner.nextInt();
                    System.out.println("Resultado: " + calc.sumar(a, b));
                }
                case 2 -> {
                    System.out.print("Ingresa primer número: ");
                    a = scanner.nextInt();
                    System.out.print("Ingresa segundo número: ");
                    b = scanner.nextInt();
                    System.out.println("Resultado: " + calc.restar(a, b));
                }
                case 3 -> {
                    System.out.print("Ingresa un número: ");
                    a = scanner.nextInt();
                    System.out.println("Valor absoluto: " + calc.absoluto(a));
                }
                case 4 -> {
                    System.out.println("Saliendo...");
                    scanner.close();
                    return;
                }
                default -> System.out.println("Opción inválida.");
            }
        }
    }
}
```

Funciones y Métodos

Las funciones son un conjunto de líneas de código (instrucciones), encapsuladas en un bloque, usualmente reciben parámetros, cuyos valores utilizan para efectuar operaciones y adicionalmente retornan un valor. En Java las funciones se llaman métodos. Estos métodos pueden usarse referidos a un objeto, es decir, necesitamos crear un objeto para usar el método. O hay métodos que no necesitan de la construcción de un objeto para su uso, estos métodos se llaman ESTÁTICOS (ver explicación en la Introducción).

Por ejemplo, cuando usamos la clase main usamos `static` porque ese método no va asociado a ningún objeto.

Sin embargo en el siguiente ejemplo necesitaremos construir un objeto de tipo Triangulo para poder calcular su área y, por tanto, no usamos `static` con el método.

Definición de Métodos

Las funciones se definirán de la siguiente manera:

```
modificadores (static) tipo_devuelto nombre_función (tipo var1, ..., tipo varn) { return expresión (del tipo devuelto); }
```

Donde:

- **Modificadores**: Los modificadores definen las características de visibilidad de la función así:
  - `private`: únicamente la clase puede acceder a la propiedad o método.
  - Package private (valor por defecto si no se indica ninguno): solo las clases en el mismo paquete pueden acceder a la propiedad o método.
  - `protected`: las clases del mismo paquete y que heredan de la clase pueden acceder a la propiedad o método.
  - `public`: la propiedad o método es accesible desde cualquier método de otra clase.
- **static**: Se usa para indicar que el método pertenece a la clase y no al objeto. Esto quiere decir que NO tendremos que crear un objeto para usar ese método, es más, no se puede usar referidos a objetos.
- **tipo_devuelto**: Si la función devuelve algo, hay que especificar el tipo del valor devuelto (int, boolean, String, array, etc). Si la función no devuelve nada se pone `void`.
- **nombre_función**: En Java se pueden escribir funciones con el mismo nombre mientras tenga diferente número de parámetros, diferente tipo o ambas (sobrecarga, explicado en la Introducción).
- **return**: En el caso de las funciones que devuelven algo se pone `return` antes de la expresión o variable a devolver. El tipo de este debe coincidir con el tipo_devuelto. NOTA: Cualquier instrucción que se encuentre después de la ejecución de `return` NO será ejecutada. Cuando es `void` no hace falta usar `return` pero se puede usar para definir dónde acaba la función.

Constructores y Encapsulación

Constructores

Un **constructor** es un método especial que se ejecuta automáticamente cada vez que se crea un objeto con `new`. Sus características son:

- Tiene el mismo nombre que la clase.
- No tiene tipo de retorno (ni siquiera `void`).
- Puede haber varios constructores en la misma clase (sobrecarga de constructores).

```
public class Triangulo {
    private long base;
    private long altura;

    // Constructor: se llama automáticamente al hacer new Triangulo(...)
    public Triangulo(long base, long altura) {
        this.base = base;
        this.altura = altura;
    }

    public long area() {
        return (base * altura) / 2;
    }
}

// Uso:
Triangulo t = new Triangulo(5, 3);
System.out.println(t.area());  // 7
```

`this`

`this` es una referencia al objeto actual, es decir, a la instancia que está ejecutando el método en ese momento. Sus usos principales son:

- **Distinguir atributo de parámetro** cuando tienen el mismo nombre: `this.base = base;`
- **Llamar a otro constructor** de la misma clase desde un constructor: `this(param1, param2);`

```java
public class Lavadora {
    private String marca;
    private boolean aguaCaliente;

    // Constructor completo
    public Lavadora(String marca, boolean aguaCaliente) {
        this.marca = marca;              // this.marca = atributo, marca = parámetro
        this.aguaCaliente = aguaCaliente;
    }

    // Constructor simplificado que delega en el completo
    public Lavadora(String marca) {
        this(marca, false);  // llama al constructor de arriba con aguaCaliente = false
    }
}
```

Encapsulación

La encapsulación consiste en proteger los atributos de una clase del acceso externo directo, declarándolos como `private` y exponiendo su lectura y modificación únicamente a través de métodos públicos llamados **getters** y **setters**. Esto permite controlar y validar los datos antes de asignarlos.

- **Getter**: devuelve el valor del atributo. Nombre: `getNombreAtributo()`.
- **Setter**: asigna un valor al atributo, pudiendo incluir validaciones. Nombre: `setNombreAtributo(valor)`.

```java
public class Electrodomestico {
    private String marca;
    private double potencia;

    public Electrodomestico(String marca, double potencia) {
        this.marca = marca.isEmpty() ? "Desconocida" : marca;
        this.potencia = Math.abs(potencia);  // validación en constructor
    }

    // Getters
    public String getMarca() { return marca; }
    public double getPotencia() { return potencia; }

    // Setters con validación
    public void setMarca(String marca) {
        this.marca = marca.isEmpty() ? "Desconocida" : marca;
    }
    public void setPotencia(double potencia) {
        this.potencia = Math.abs(potencia);
    }
}
```

Métodos, Funciones y Clases Avanzadas

Paquetes y Organización de Clases

En Java, los paquetes (packages) permiten organizar clases y evitar conflictos de nombres. Un paquete es como una carpeta que agrupa clases relacionadas. Se declara al inicio del archivo con `package nombrePaquete;`.

Para usar clases de otros paquetes, se utiliza `import nombrePaquete.NombreClase;` o `import nombrePaquete.*;` para importar todas las clases del paquete.

Ejemplo: En un proyecto de parking, podrías tener clases en paquetes separados:

```java
// En parking/DatosParking.java
package parking;

public class DatosParking {
    public static int plazasLibres(int total, int ocupadas) {
        return total - ocupadas;
    }
}

// En calculos/CalculosParking.java
package calculos;

public class CalculosParking {
    public static double porcentajeOcupacion(int total, int ocupadas) {
        return ((double) ocupadas / total) * 100.0;
    }
}
```

```java
// En parking/ProgramaPrincipal.java
package parking;

import calculos.CalculosParking;  // Importa la clase específica

public class ProgramaPrincipal {
    public static void main(String[] args) {
        int libres = DatosParking.plazasLibres(100, 20);
        double porcentaje = CalculosParking.porcentajeOcupacion(100, 20);
        System.out.println("Plazas libres: " + libres + ", Porcentaje: " + porcentaje + "%");
    }
}
```

Esto facilita la modularidad y reutilización en proyectos POO.

Llamadas entre Métodos y Clases

En POO, los métodos pueden llamar a otros métodos dentro de la misma clase o en clases diferentes. Métodos estáticos llaman a otros estáticos sin necesidad de instancia.

Ejemplo de llamadas en la misma clase (expandiendo DatosParking):

```java
public class DatosParking {
    public static int plazasLibres(int total, int ocupadas) {
        return total - ocupadas;
    }

    public static String estadoParking(int total, int ocupadas) {
        if (plazasLibres(total, ocupadas) == 0) {  // Llama a otro método estático
            return "COMPLETO";
        } else if (plazasLibres(total, ocupadas) <= 5) {
            return "CASI COMPLETO";
        } else {
            return "DISPONIBLE";
        }
    }
}
```

Para clases diferentes, importa el paquete y llama directamente (como en ProgramaPrincipal llamando a CalculosParking).

Manipulación de Cadenas (Strings)

Las cadenas en Java son objetos inmutables. Métodos útiles:

- `length()`: Longitud.
- `charAt(int index)`: Carácter en posición.
- `substring(int begin, int end)`: Subcadena.
- `split(String regex)`: Divide en array (ej. `frase.split("\\s+")` para palabras).
- `trim()`: Elimina espacios extremos.
- `replaceAll(String regex, String replacement)`: Reemplaza (ej. quitar espacios: `replaceAll("\\s+", "")`).
- `toLowerCase()` / `toUpperCase()`: Cambia caso.
- `equalsIgnoreCase(String another)`: Compara ignorando caso.
- `indexOf(String str)`: Posición de substring.

Ejemplo (de análisis de texto):

```java
public class ManipulacionStrings {
    public static int contarPalabras(String frase) {
        if (frase == null || frase.isEmpty()) return 0;
        String[] palabras = frase.trim().split("\\s+");
        return palabras.length;
    }

    public static boolean esPalindromo(String palabra) {
        String limpia = palabra.replaceAll("\\s+", "").toLowerCase();
        int izq = 0, der = limpia.length() - 1;
        while (izq < der) {
            if (limpia.charAt(izq) != limpia.charAt(der)) return false;
            izq++; der--;
        }
        return true;
    }

    public static void main(String[] args) {
        String texto = "Hola mundo cruel";
```

```
        System.out.println("Palabras: " + contarPalabras(texto));
        System.out.println("Es palíndromo 'ana': " + esPalindromo("ana"));
    }
}
```

Uso de Librerías Estándar

Java incluye librerías útiles importadas automáticamente o con `import`.

- **Scanner**: Para entrada de usuario. `import java.util.Scanner;`. Ejemplo: `Scanner sc = new Scanner(System.in); int num = sc.nextInt();`.
- **Math**: Funciones matemáticas. Ejemplo: `Math.random()` para aleatorio (0.0 a 1.0), `Math.abs(int)` para valor absoluto.
- **Character**: Para caracteres. Ejemplo: `Character.isDigit(char)` para verificar dígitos.
- **LocalDate**: Para trabajar con fechas. `import java.time.LocalDate;`. Ejemplo: `LocalDate.now()` para la fecha actual, `LocalDate.of(2024, 3, 15)` para una fecha concreta, `fecha.isAfter(otraFecha)` para comparar.

Ejemplo en un menú:

```
import java.util.Scanner;

public class MenuEjemplo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Elige: 1-Suma, 2-Resta");
        int opcion = sc.nextInt();
        switch (opcion) {
            case 1 -> {
                System.out.print("Num1: ");
                int a = sc.nextInt();
                System.out.print("Num2: ");
                int b = sc.nextInt();
                System.out.printf("Suma: %d%n", a + b);  // Formateo con printf
            }
            case 2 -> {
                // Similar
            }
            default -> System.out.println("Opción inválida");
        }
        sc.close();
    }
}
```

Usa `printf` para formateo: `%d` enteros, `%f` floats, `%s` strings, `%n` nueva línea.

Excepciones Básicas

Las excepciones manejan errores en tiempo de ejecución. Se lanzan con `throw new TipoExcepcion("mensaje");`. Tipos comunes: `IllegalArgumentException` para argumentos inválidos.

Ejemplo en método:

```
public static boolean esDivisible(int a, int b) {
    if (b == 0) {
        throw new IllegalArgumentException("El divisor no puede ser cero.");
    }
    return a % b == 0;
}
```

Extra: Manejo con try-catch

En `main`, envuélvelo en `try-catch` para manejar:

```
try {
    boolean resultado = esDivisible(10, 0);
} catch (IllegalArgumentException e) {
    System.out.println("Error: " + e.getMessage());
}
```

Esto previene crashes y mejora robustez en POO.

Arrays en Java

Los arrays son estructuras de datos que almacenan múltiples valores del mismo tipo. Se declaran con `tipo[] nombreArray = new tipo[tamaño];` o con inicialización directa `tipo[] nombreArray = {valor1, valor2};`.

- **Arrays 1D**: Para listas simples, como `int[] numeros = new int[5];`.
- **Arrays Multidimensionales**: Para tablas, como `String[][] contactos = new String[10][2];` (10 filas, 2 columnas).

Acceso: `array[indice]` (índices empiezan en 0). Pueden contener `null` si no se inicializan.

Ejemplo de manipulación (basado en agenda de contactos):

```
public class EjemploArrays {
    public static void main(String[] args) {
        // Array 1D paralelo
        String[] nombres = new String[10];
        String[] telefonos = new String[10];
        int total = 0;

        // Agregar contacto
        nombres[total] = "Juan";
        telefonos[total] = "123456";
        total++;

        // Buscar (ignorando mayúsculas)
        String buscar = "juan";
        for (int i = 0; i < total; i++) {
            if (nombres[i] != null && nombres[i].equalsIgnoreCase(buscar)) {
                System.out.println("Teléfono: " + telefonos[i]);
                break;
            }
        }

        // Array 2D
        String[][] contactos2D = new String[10][2];  // [fila][columna]: 0=nombre, 1=teléfono
        contactos2D[0][0] = "Ana";
        contactos2D[0][1] = "789012";
        System.out.println("Nombre: " + contactos2D[0][0] + ", Teléfono: " + contactos2D[0][1]);
    }
}
```

Los arrays se recorren con bucles `for` o `for-each`.

Extra: Bucles for-each

El bucle `for-each` simplifica el recorrido de arrays:

```
int[] array = {1, 2, 3};
for (int num : array) {
    System.out.print(num + " ");
}
```

Esto imprime: 1 2 3

Herencia

Podemos definir la herencia como la capacidad de crear clases que adquieren de manera automática los miembros (atributos y métodos) de otras clases que ya existen, pudiendo al mismo tiempo añadir atributos y métodos propios.

Otra forma de verlo es que podemos sacar factor común en varias clases para optimizar y minimizar el código. Por ejemplo, si tenemos las clases `Profesor` y `Alumno`, ambas comparten atributos como nombre, apellidos y contacto. En lugar de repetirlos, podemos crear una clase `Persona` que los contenga, y hacer que ambas la hereden.

La clase que hereda se llama **subclase** o clase derivada, y la clase de la que se hereda se llama **superclase** o clase base. En Java, la herencia se indica con la palabra clave `extends`.

```
class Vehiculo {
    protected String marca;
    protected String modelo;
    protected int anio;

    public Vehiculo(String marca, String modelo, int anio) {
        this.marca = marca;
        this.modelo = modelo;
        this.anio = anio;
    }

    public String descripcion() {
```

```
        return marca + " " + modelo + " (" + anio + ")";
    }
}

class Coche extends Vehiculo {
    private int puertas;

    public Coche(String marca, String modelo, int anio, int puertas) {
        super(marca, modelo, anio);  // Llama al constructor de Vehiculo
        this.puertas = puertas;
    }

    @Override
    public String descripcion() {
        return super.descripcion() + " - Puertas: " + puertas;
    }
}
```

`super`

`super` permite acceder a los miembros de la superclase desde la subclase:

- `super(params)`: llama al constructor de la superclase. Debe ser siempre la **primera instrucción** del constructor hijo.
- `super.metodo()`: llama a la versión del método que está definida en la superclase.

Sobrescritura de métodos (`@Override`)

Cuando una subclase redefine un método que ya existía en la superclase, se dice que lo **sobrescribe**. La anotación `@Override` es opcional pero muy recomendable: le indica al compilador que la intención es sobrescribir, y este verificará que la firma del método coincida exactamente con la del padre, evitando errores silenciosos.

Los atributos `private` de la superclase no son accesibles directamente en la subclase; se deben usar los getters y setters. Los atributos `protected` sí son accesibles directamente desde las subclases.

Tipos de herencia

En Java existen varios tipos de herencia:

- **Herencia única**: una subclase hereda de una sola superclase. Es el caso más común. Ejemplo: `Coche extends Vehiculo`.
- **Herencia Multinivel**: una clase derivada hereda de otra clase que a su vez es derivada de otra. Se forma una cadena. Ejemplo: `Camion extends Vehiculo`, y después `CamionArticulado extends Camion`.
- **Herencia Jerárquica**: varias subclases heredan de la misma superclase. Ejemplo: `Coche`, `Camion` y `Moto` heredan todas de `Vehiculo`.
- **Herencia Múltiple**: una clase hereda de varias superclases a la vez. **Java no permite herencia múltiple entre clases**. Para conseguir un efecto equivalente se utilizan las **interfaces** (ver sección siguiente).

Clases Abstractas

Una **clase abstracta** es una clase que no puede instanciarse directamente, es decir, no se puede hacer `new ClaseAbstracta()`. Su propósito es servir de base común para otras clases, definiendo atributos y métodos compartidos, y obligando a las subclases concretas a implementar ciertos métodos.

Se declara con la palabra clave `abstract` antes de `class`. Puede contener:

- **Métodos concretos**: con implementación completa, que las subclases heredan tal cual.
- **Métodos abstractos**: declarados sin cuerpo, terminan en `;`. Las subclases concretas **deben** implementarlos obligatoriamente.

```
public abstract class FiguraGeometrica {
    protected String nombre;
    protected String color;

    // Constructor (las subclases lo llaman con super)
    public FiguraGeometrica(String nombre, String color) {
        this.nombre = nombre;
        this.color = color;
    }

    // Método concreto: implementación compartida por todas las figuras
    public void mostrarInformacion() {
        System.out.println("Figura: " + nombre + " (color=" + color + ")");
    }

    // Métodos abstractos: cada subclase los implementa a su manera
    public abstract double calcularArea();
    public abstract double calcularPerimetro();
    public abstract void dibujar();
}
```

Las subclases concretas deben implementar todos los métodos abstractos:

```
public class Circulo extends FiguraGeometrica {
    private double radio;

    public Circulo(String nombre, String color, double radio) {
        super(nombre, color);
        this.radio = radio;
    }

    @Override
    public double calcularArea() { return Math.PI * radio * radio; }

    @Override
    public double calcularPerimetro() { return 2 * Math.PI * radio; }

    @Override
    public void dibujar() {
        System.out.println("Dibujando círculo de radio " + radio);
    }
}
```

Si una subclase no implementa todos los métodos abstractos del padre, debe declararse también como `abstract`.

Cuándo usar una clase abstracta: cuando varias clases comparten atributos y código real (métodos concretos), pero también necesitan comportamiento específico obligatorio que cada una debe definir por su cuenta.

Interfaces

En el punto anterior hemos hablado de las clases abstractas y de la herencia múltiple, conceptos necesarios para entender qué es una interfaz y para qué sirve. Una interfaz no es más que una clase donde todos sus métodos son abstractos y solo tiene constantes estáticas.

Las interfaces definen un protocolo de comportamiento y proporcionan un formato común para implementarlo en las clases. De esta manera una interfaz va a definir qué se debe hacer pero no cómo.

Una interfaz se crea utilizando la palabra clave `interface` en lugar de `class`:

```
public interface NombreInterface [extends Interface1, Interface2, ...] {
    // constantes y métodos abstractos
}
```

Algunas características de las interfaces:

- La interfaz puede definirse `public` o sin modificador de acceso, y tiene el mismo significado que para las clases. Si tiene el modificador `public` el archivo `.java` que la contiene debe tener el mismo nombre que la interfaz.
- En los métodos abstractos no es necesario escribir `abstract`.
- Todos los atributos son constantes, públicos y estáticos. Por lo tanto, se pueden omitir los modificadores `public static final` cuando se declara el atributo. Se deben inicializar en la misma instrucción de declaración.

```
public interface Empleado {
    double calcularSalarioMensual();  // implícitamente public abstract
    void mostrarDetalles();
}
```

Implementación de una interfaz

Para implementar una interfaz se usa la palabra clave `implements`. La clase debe proporcionar una implementación para todos los métodos de la interfaz:

```
public class EmpleadoRegular implements Empleado {
    private String nombre;
    private double salarioBase;

    public EmpleadoRegular(String nombre, double salarioBase) {
        this.nombre = nombre;
        this.salarioBase = salarioBase;
    }

    @Override
    public double calcularSalarioMensual() { return salarioBase; }

    @Override
    public void mostrarDetalles() {
```

```
        System.out.println(nombre + " | Salario: " + calcularSalarioMensual());
    }
}
```

Múltiples interfaces

Una clase puede implementar varias interfaces a la vez, separándolas por comas. Esto es la forma en que Java consigue herencia múltiple:

```
public class Robot implements Encendible, ConectableWifi {
    @Override public void encender() { System.out.println("Robot encendido"); }
    @Override public void conectar(String red) { System.out.println("Conectado a " + red); }
}
```

Clase abstracta que implementa una interfaz

Una clase abstracta puede implementar una interfaz sin estar obligada a implementar todos sus métodos. Los métodos que no implemente quedarán pendientes para que los resuelvan sus subclases concretas:

```
public abstract class EmpleadoBase implements Empleado {
    protected String nombre;
    protected int id;
    protected double salarioBase;

    public EmpleadoBase(String nombre, int id, double salarioBase) {
        this.nombre = nombre;
        this.id = id;
        this.salarioBase = salarioBase;
    }

    @Override
    public void mostrarDetalles() {  // implementa este método de la interfaz
        System.out.println("Nombre: " + nombre + " | ID: " + id);
    }
    // calcularSalarioMensual() queda sin implementar → lo resuelve cada subclase
}

public class Gerente extends EmpleadoBase {
    private double bonoAnual;

    public Gerente(String nombre, int id, double salarioBase, double bonoAnual) {
        super(nombre, id, salarioBase);
        this.bonoAnual = bonoAnual;
    }

    @Override
    public double calcularSalarioMensual() { return salarioBase + bonoAnual; }
}
```

Métodos `default` (Java 8+)

A partir de Java 8, las interfaces pueden incluir métodos con implementación usando la palabra clave `default`. Las clases que implementen la interfaz pueden usar ese método tal cual o sobrescribirlo:

```
public interface Vehiculo {
    default void arrancar() {
        System.out.println("Arrancando...");
    }
}
```

Polimorfismo

El polimorfismo es la capacidad de una variable del tipo de la superclase o de una interfaz de referirse a objetos de distintas subclases, ejecutando en cada caso el método correspondiente a la clase real del objeto. Esto permite escribir código genérico que funciona con cualquier subclase sin necesidad de conocerla de antemano.

```
FiguraGeometrica[] figuras = {
    new Circulo("c1", "rojo", 5.0),
    new Rectangulo("r1", "azul", 4.0, 3.0)
};

for (FiguraGeometrica f : figuras) {
    System.out.println(f.calcularArea());  // ejecuta el método de cada clase real
```

```
}

// Con interfaces:
Empleado e = new Gerente("Ana", 1, 2000, 500);
e.calcularSalarioMensual();  // ejecuta Gerente.calcularSalarioMensual()
```

Enumerados (`enum`)

Un `enum` define un conjunto fijo y cerrado de constantes con nombre. Se utilizan para representar valores que solo pueden tomar una de varias opciones predefinidas, evitando el uso de Strings libres que pueden dar lugar a errores tipográficos.

```
public enum TipoElectrodomestico {
    NEVERA, HORNO, LAVADORA, MICROONDAS, SECADORA
}
```

Se suelen declarar dentro de la clase que los usa o en su propio archivo. Para usarlos:

```
TipoElectrodomestico tipo = TipoElectrodomestico.LAVADORA;

// En un constructor con validación:
public Electrodomestico(TipoElectrodomestico tipo, String marca, double potencia) {
    this.tipo = (tipo == null ? TipoElectrodomestico.HORNO : tipo);
    this.marca = marca;
    this.potencia = potencia;
}
```